

Use of Find operator

Q. Return the intersection of two linked list
list1 $\rightarrow 9 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 8 \rightarrow 8$
list2 $\rightarrow 1 \rightarrow 2 \rightarrow 8 \rightarrow 6$

Note: Result should be in order of first list;

→ Node* FindIntersection(Node* head1, Node* head2)

{ unordered_set<int> s;

while (head2 != NULL)

{ s.insert(head2->data);

head2 = head2->next;

Node* start = NULL, *end = NULL;

while (head1 != NULL)

{ if (s.find(head1->data) != s.end())

{ if (start == NULL)

{ start = end = head1;

}

else

{ end->next = head1;

end = head1;

}

head1 = head1->next;

}

if (start != NULL)

{

end->next = NULL;

}

return start;

}

Note:- while getting segmentation error, think simple testcase you can think of.

Linked List

Page No.

Date: / /

1. Problem with Arrays
2. Difference b/w Arrays and Linked List

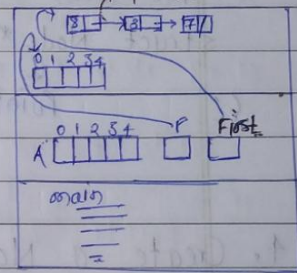
1. Problem with Arrays 2. Difference b/w Arrays and Linked List

1. Array is of Fixed size (Once created)
2. Due to Fixed size either there is shortage of memory or there is lack of memory

A[10]

int A[5];

int *P = new int[5];



1. What is Linked List

2. What is a Node

3. Node structure

4. Create a Node

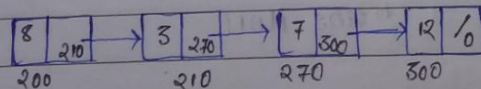
3. Access Node

1. What is Linked List → Created inside heap

First

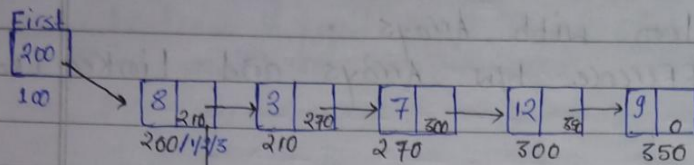
200 → called as Head node or First node

100

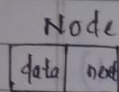


Linked List is a collection of node where each node contains data & pointer to next node

2.3. Node & Node structure



Pointer is of node type for next node



SELF - Referential Pointer

```

struct Node
{
    int data;
    struct Node *next;
};
  
```

2 bytes

2 bytes

4 bytes

↓
Pointer of structure node

Every node consume 4 byte

4. Create a Node

```
struct Node *P;
```

```
c -> P = (struct Node *) malloc (sizeof (struct Node));
```

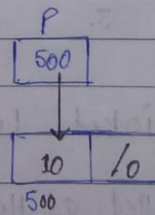
```
c++ -> P = new Node;
```

5. Access a Node

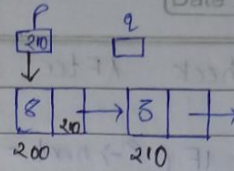
```
P -> data = 10;
```

```
P -> next = 0;
```

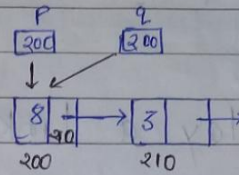
↓
means NULL



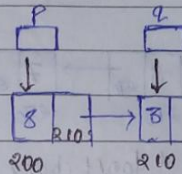
→ struct Node *P, *q;



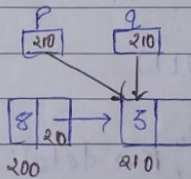
① $q = P;$



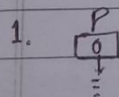
② $q = P \rightarrow \text{next};$



③ $P = P \rightarrow \text{next};$



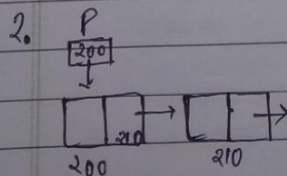
→ struct Node *P = NULL / struct Node *P = 0



→ IF (P == NULL)

IF (P == 0)

IF (!P)



→ IF (P != NULL)

IF (P != 0)

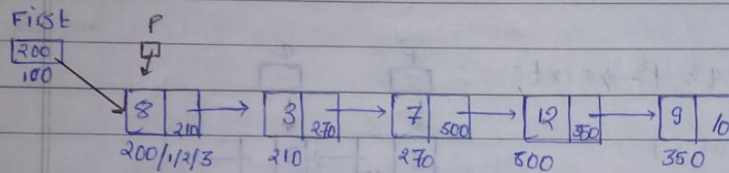
IF (P)

⑤ To check After one node next node is present or not

IF (P->next == NULL)

IF (P->next != NULL)

How to display a Linked List



struct Node *P = First;

while (P != NULL)

printf("%d", P->data);
P = P->next;

display(struct Node *P)

while (P != NULL)

printf("%d", P->data);
P = P->next;

display(First);

#include <stdio.h>
#include <stdlib.h>

struct Node

{
int data;

struct Node *next;

*first = NULL

↓
Global Pointer (can be directly accessed or passed as pointer)


```
→ void create (int A[], int n)
```

```
{ int i;
```

```
  struct Node *t, *last;
```

```
  First = (struct Node *) malloc (sizeof (struct Node));
```

```
  First → data = A[0];
```

```
  First → next = NULL;
```

```
  last = First;
```

```
  for (i = 1; i < n; i++)
```

```
{
```

```
    t = (struct Node *) malloc (sizeof (struct Node));
```

```
    t → data = A[i];
```

```
    t → next = NULL;
```

```
    last → next = t;
```

```
    last = t;
```

```
}
```

```
→ void display (struct Node *P)
```

```
{
```

```
  while (P != NULL)
```

```
{
```

```
    printf ("%d", P → data);
```

```
    P = P → next;
```

```
}
```

```
}
```

```
int main ()
```

```
{ int A[] = { 3, 5, 7, 10, 15 };
```

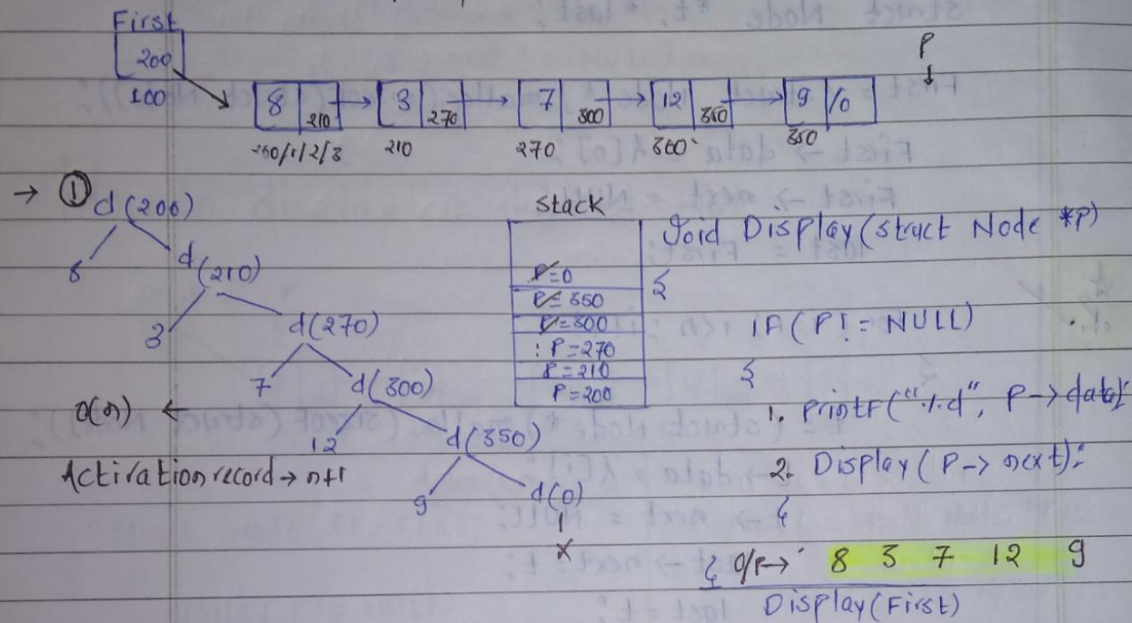
```
  create (A, 5);
```

```
  display (first);
```

```
→ 3, 5, 7, 10, 15
```

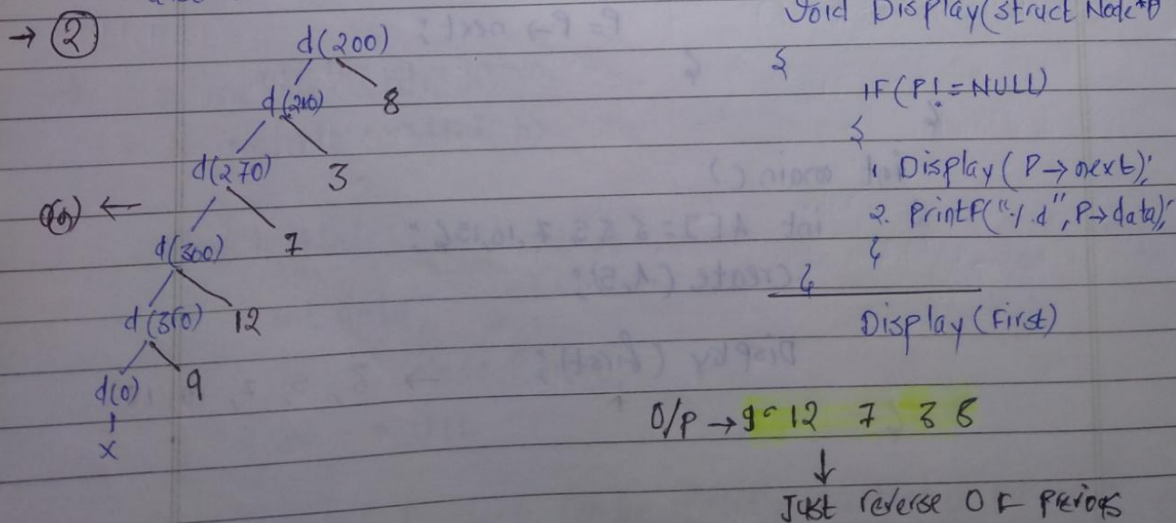
```
}
```

Recursive Display OF Linked List

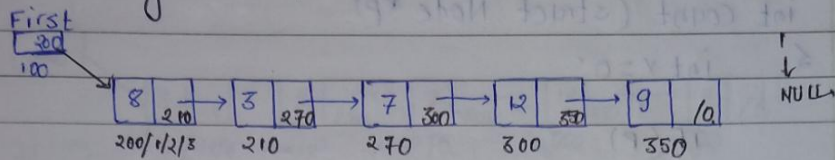


* IF any procedure traversing the linked list once then time will be order of n only

* IF any procedure traversing a linked list once then it will make n+1 call for entire linked list & stack size will also be n+1



Counting Nodes in a Linked List



```

1) int count(struct Node *P)
   {
       int c = 0;
       while (P != 0)
       {
           c++;
           P = P->next;
       }
       return c;
   }
  
```

→ $O(n)$ → Time
 → $O(1)$ → Space

2)

```

int count(struct Node *P)
{
    IF (P != NULL) IF (P == 0)
    return 1 + count(P->next); return 0;
    else
    return 0;
    else
    return count(P->next) + 1;
}
  
```

Time - $O(n)$
 Space - $O(n)$

Recursion tree for counting nodes:

```

graph TD
    C200["C(200) = 5"] --> C210["C(210) + 1 = 4"]
    C210 --> C270["C(270) + 1 = 3"]
    C270 --> C300["C(300) + 1 = 2"]
    C300 --> C350["C(350) + 1 = 1"]
    C350 --> C0["C(0) + 1 = 0"]
    C0 --> 0["0"]
  
```

★ Addition will be done during returning time only

This one will be added during returning

This + will be added when C(210) returns

[OR]

```
int count (struct Node *p)
```

```
{
    int x = 0;
```

```
    IF (P)
```

```
    {
        x = count (P->next);
```

```
        return x+1;
```

```
    }
```

```
    else
```

```
    {
```

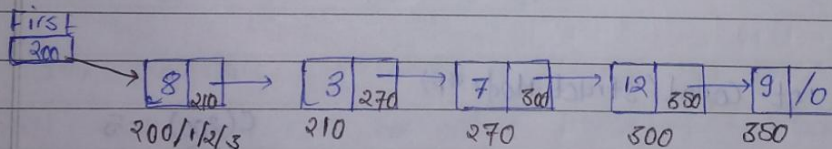
```
        return 0;
```

```
    } or return x;
```

```
}
```

* if coming directly in else part then x is 0 only

Sum OF all element in a Linked list



Add(First)

```
int Add(struct Node *p)
```

```
{
    int sum = 0;
```

```
    While (P)
```

```
    {
```

```
        sum = sum + P->data;
```

```
        P = P->next;
```

```
    }
```

```
    return sum;
```

```
}
```

```
int Add(struct Node *p)
```

```
{
```

```
    IF (P == 0)
```

```
        return 0;
```

```
    else
```

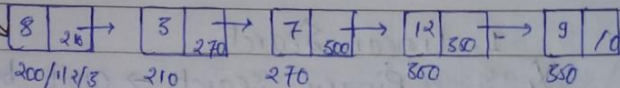
```
        return Add(P->next) + P->data;
```

```
}
```


Maximum element in a Linked List

First

200



max = MIN-INT

max = -32768

Recursive

int max (Node *P)

int m = -32768; MIN-INT;

while (P)

if (P->data > m)

m = P->data;

P = P->next;

return (m);

int max (Node *P)

{

int x = 0;

if (P == 0)

return MIN-INT;

else

x = max (P->next);

if (x > P->data)

return x;

else

return P->data;

}

OR

int max (Node *P)

{

int x = 0;

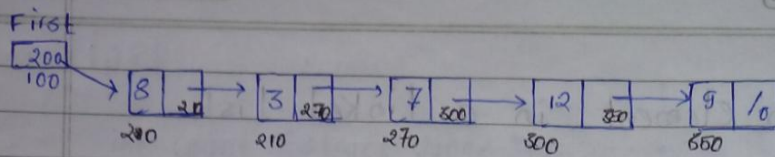
if (P == 0)

return MIN-INT;

x = Max (P->next);

return x > P->data ? x : P->data;

ternary operator



1. Linear Search

X 2. Binary search → can't be performed on linked list, we can't directly reach in middle

Node * search(Node* P, int key)

```

while ( P != NULL )
{
    IF ( key == P->data )
        return (P);

    P = P->next;
}
return NULL;

```

Node * search(Node* P, int key)

```

IF ( P == NULL )
    return NULL;

IF ( key == P->data )
    return (P);

return search( P->next, key );

```

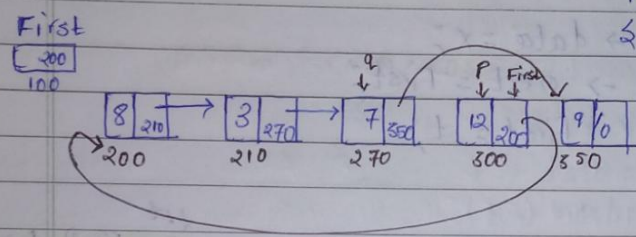
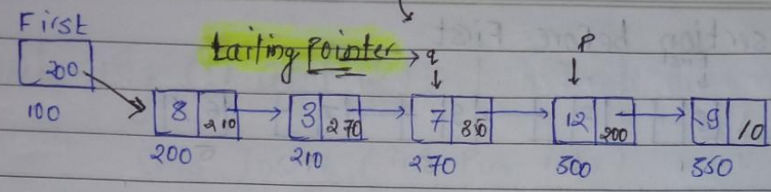
Proving Searching in a Linked List

① Transposition → with previous value swap

② Move to head → snapping with First position

★ we avoid movement of data in linked list so we avoid Transposition Method, we prefer Movement of Node

* having pointer q following P



Node *search (Node *P, int Key)

Node *q = NULL;

while (P != NULL)

if (Key == P->data)

q->next = P->next;

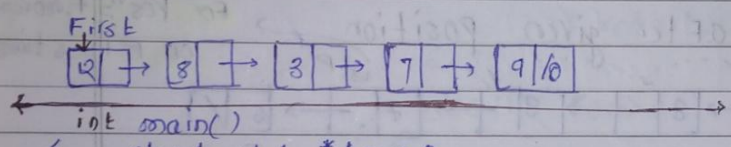
P->next = First;

First = P;

q = P;

P = P->next;

Pointer q
Following P



int main()

struct Node *temp;

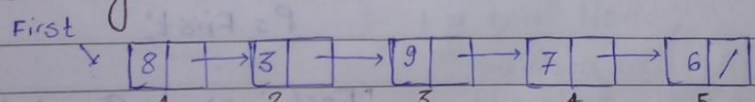
temp = lsearch (First, 25);

if (temp) printf("Key Found");

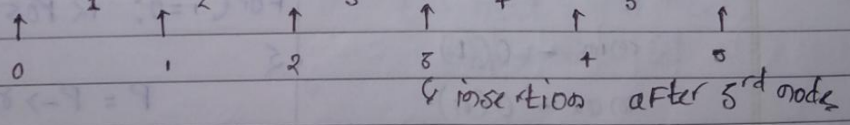
else printf("Key not Found");

Display (First);

Inserting in a Linked list

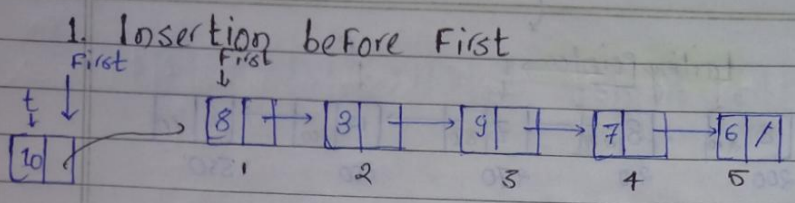


→ Posⁿ:-

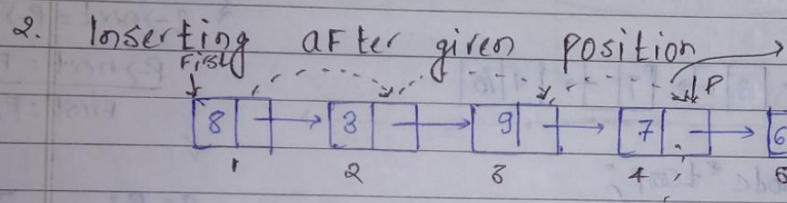


1. Insertion before First

2. Inserting after given position



Node *t = New Node
 $t \rightarrow \text{data} = x;$
 $t \rightarrow \text{next} = \text{First};$
 $\text{First} = t;$



For Posth = 4, movement of P is 3 times

Posth = 4
 Node *t = new Node;
 $t \rightarrow \text{data} = x;$
 $P = \text{First};$

time - $O(L)$
 space - $O(1)$

For (i=0; i < Pos-1; i++)
 $P = P \rightarrow \text{next};$
 $t \rightarrow \text{next} = P \rightarrow \text{next};$
 $P \rightarrow \text{next} = t;$

Combining ① and ②

Void Insert (int Pos, intx)

Node *t, *P;

IF (Pos == 0)

t = new Node;

t->data = x;

t->next = First;

First = t;

{

else IF (Pos > 0)

P = First;

For (i = 0; i < Pos - 1 && P != NULL; i++)

P = P->next;

IF (P == NULL)

t = new Node;

t->data = x;

t->next = P->next;

P->next = t;

{

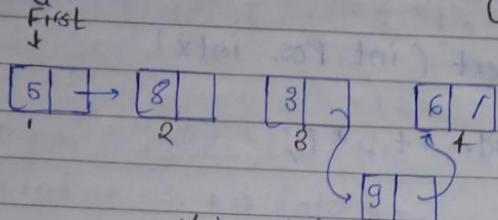
}

}

P != NULL

*** For checking
P is null or not

* Creating a Linked List Using Insert



X insert(3, 7) → invalid indexes

* visualise for creation

insert(0, 8):

insert(1, 3):

insert(2, 6):

insert(0, 5):

insert(3, 9):

* Always make sure you are entering valid index for creating linked list

* Creating a Linked List by inserting at last

Case ①

First last → no node → We have to handle this situation also

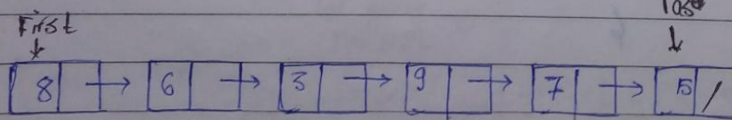
Case ②

8 → one node

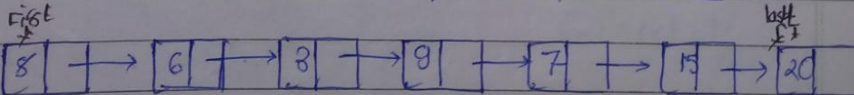
Case ③

8 → 12 → 2 node

We have to deal above three cases for inserting in last



After inserting at last



void insertlast (int x)

{

Node *t = new Node;

t->data = x;

t->next = NULL;

IF (First == NULL)

{

First = last = t;

}

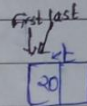
else

{ last->next = t;

last = t;

}

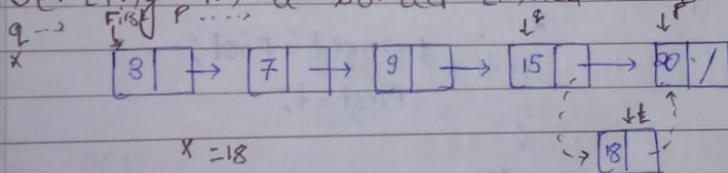
}



→ IF there is no node

IF node present inserting at last

Inserting in a Sorted Linked List



* * q is called
tail pointer

P = First;

q = NULL;

while (P && P->data < x)

{

q = P;

P = P->next;

}

t = new Node;

t->data = x;

t->next = q->next;

q->next = t;

→ void SortedInsert(struct Node* P, int x)

{

struct Node *t, *q = NULL;

t = (struct Node*) malloc(sizeof(struct Node));

t->data = x;

t->next = NULL;

IF no element Present → IF (First == NULL)

First = t;

else

{ while (P && P->data < x)

{

q = P;

P = P->next;

}

IF only First present → IF (P == First)

{

t->next = First;

First = t;

}

else

{

t->next = q->next;

q->next = t;

}

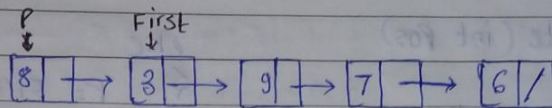
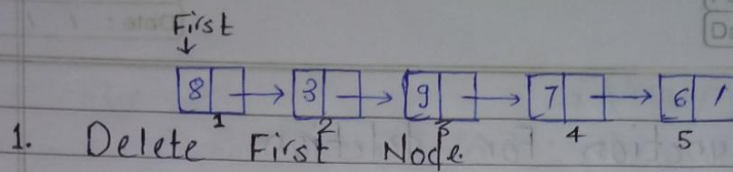
}

}

Deleting From Linked List

1. Delete First Node

2. Delete a Node at given Position



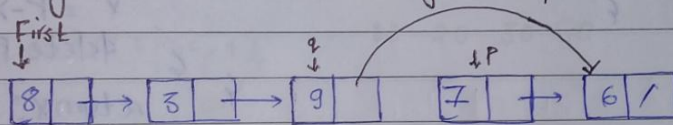
Node *P = First;

First = First->next;

Time - $O(1)$

x = P->data; → before deletion
delete P; take the value x
then delete it

2. Deleting a node at given position.



Node *P = First;

Node *q = NULL;

min - $O(1)$

max - $O(n)$

For (i = 0; i < pos - 1; i++)

{ q = P;

P = P->next; }

{

q->next = P->next;

x = P->data;

delete P;

Single Function For deletion

```

int Delete(int Pos)
{
    Node *P, *q;
    int x = -1, i = 1;
    if (Pos == 1)
    {
        x = First->data;
        P = First;
        First = First->next;
        delete P;
    }
    else
    {
        P = First;
        q = NULL;
        For (i = 0; i < Pos - 1; i++)
        {
            q = P;
            P = P->next;
        }
        if (P)
        {
            q->next = P->next;
            x = P->data;
            delete P;
        }
        return x;
    }
}

```

To check P is null or not

IF P not null
check for 6 pos
or 7 pos

```

→ int Delete(struct Node *P, int index)
{
    struct Node *q;
    int x = -1, i = 1;
    if (index < 1 || index > Count(P))
        return -1;
    if (index == 1)
    {
        q = First;
        x = First->data;
        First = First->next;
        delete q;
        return x;
    }
}

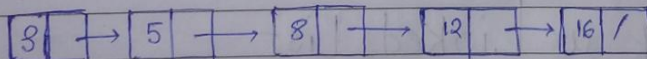
```



```
int main
```

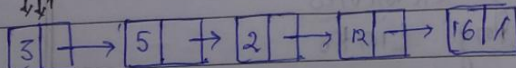
~~Delete(First, +) → 10 20 30 50~~

First
↓


$$X = -32768$$

8
8
8
~~12~~

first
↓ ↓ ↑


$$x = -32768$$

3
8
1

NON return false

Sorted

$$O(n)$$

Not sorted

$$M(10) = O(1)$$
$$M(ax^n) = O(n)$$

17E $x = -32768, / -65536,$

Node * p = First;

```
while (P! = NULL)
```

2 IF (P → data < x)
return False;

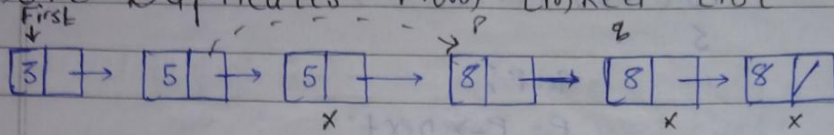
$x = P \rightarrow \text{data};$

$p = p \rightarrow next$

6

return true;

Remove Duplicates From Linked List



$O(n)$

Node *P = First;

Node *q = First->next;

While (q != NULL)

{

IF (P->data != q->data)

{ P = q;

q = q->next;

}

else

{ P->next = q->next;

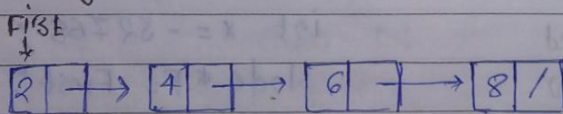
delete q;

q = P->next;

}

Also see by
tracking the code

Reversing a Linked List



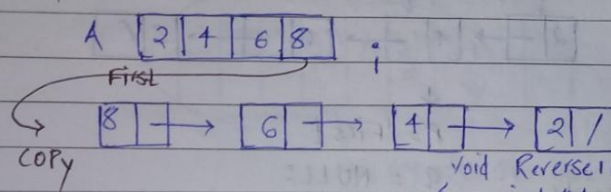
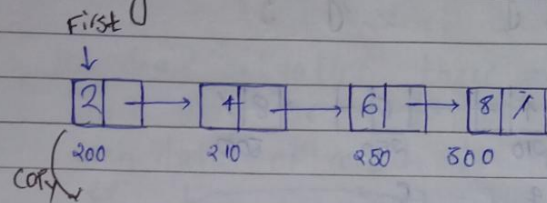
Methods:-

1. Reversing elements

2. Reversing Links

→ ** it is more preferred

1. Reversing elements



```

void Reverse1(struct Node *P)
{
    int *A, i=0; A=(int*)malloc(sizeof(int)*count(P));
  
```

P = First;

i=0;

while (P != NULL)

{ A[i] = P->data;

P = P->next;

i++;

P = First, i--;

while (P != NULL)

{ P->data = A[i--];

P = P->next;

Reverse1(First);

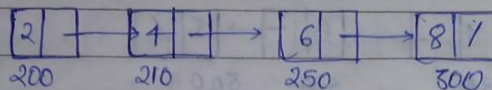
* This method takes extra space of array

$O(n)$

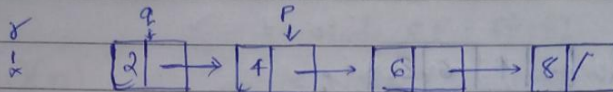
*** Q. Reversing Using sliding pointers / Reversing link

sliding
Pointer

First P



movement



P = First

Q = NULL;

r = NULL;

While (P != NULL)

{

r = Q;

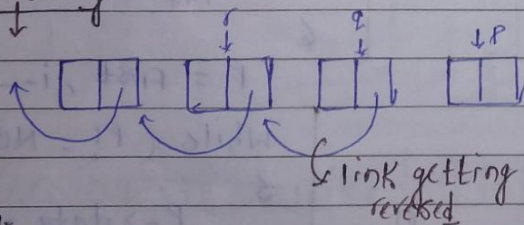
Q = P;

P = P -> next;

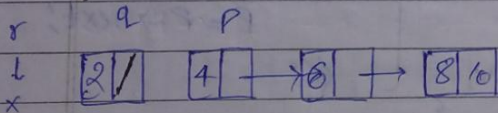
}

Code for movement
or
sliding

Reversing



Tracing



⇒ see by tracing

P = First;

Q = NULL;

r = NULL;

While (P != NULL)

{

r = Q;

Q = P;

P = P -> next;

* → Q → next = r;

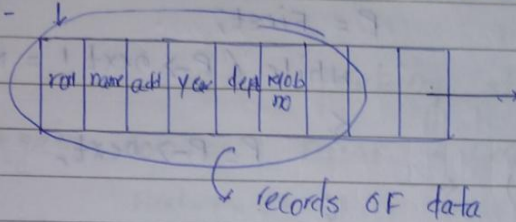
}

* First = Q;

∴ Reason For using reverse link

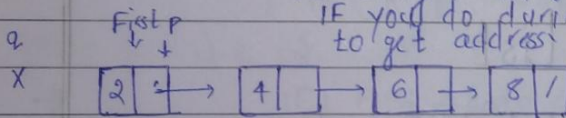
In some situation there may not be only integer there may be records of data. so swapping each data of record is consuming of time & memory

ex:-



Recursive Reverse For Linked List

reversing should be done during returning time. IF you do during going you won't be able to get address



void Reverse(Node *q, Node *P)

{

IF (P != NULL)

→ Reverse(P, P->next);

→ P->next = q;

else

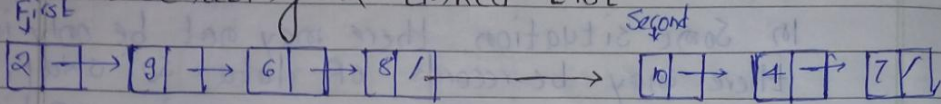
→ First = q;

reversing link during returning time

→ Reverse(NULL, First);

}

Concatenating 2 linked List



$P = \text{First}$

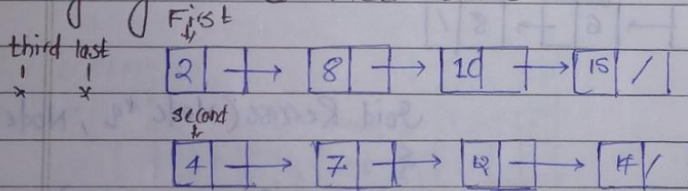
while (P->next != NULL)

2

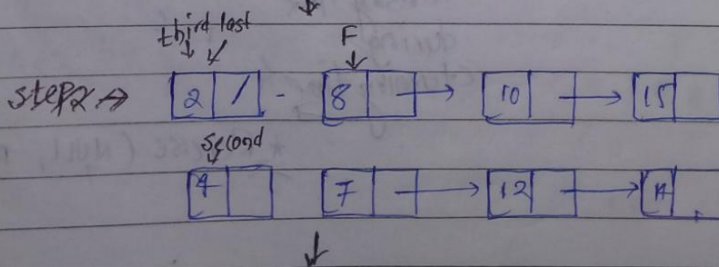
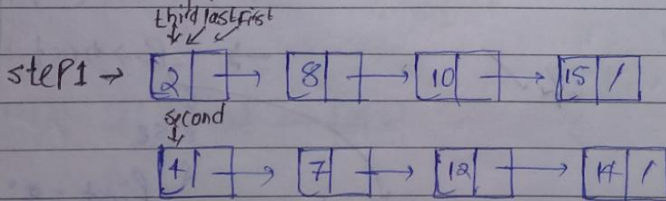
$$P = P \rightarrow \text{next},$$
$$O(n)$$
$$P \rightarrow next = second^0$$

second: NULL

★ ✓ Merging 2 Linked List

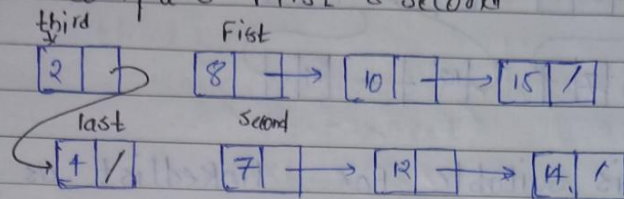


compare First & second



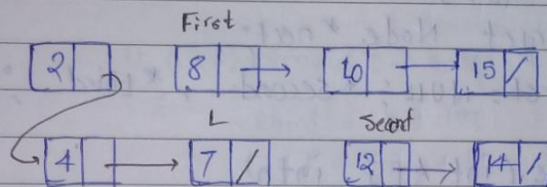
Page No.
 Date: / /

step 2 Again Compare First & second



Now repeating steps

step 3 → Again Compare First & second



IF (First → data < second → data)

→

IF (First → data < second → data)

third = last = First;
First = First → next;
Fast → next = NULL;

else
third = last = second;
second = second → next;
last → next = NULL;

While (First != NULL && second != NULL)

IF (First → data < second → data)

last → next = First; last = First; First = First → next; last → next = NULL;

else

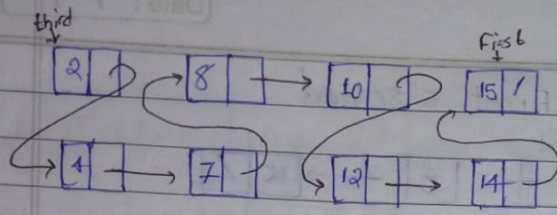
last → next = second; last = second; second = second → next; last → next = NULL;

IF (First != NULL) last → next = First

last → next = second

$O(m+n)$

to show
merging



★ Merging is suitable for linkedlist as there no extra linked list required.

Ex:-

```
struct data
{
    int data;
    struct Node *next;
}
* First = NULL; * second, * third;
```

```
void create (int A[], int n)
{
    int i;
    struct Node *t, *last;
    First = (struct Node*) malloc(sizeof(struct Node));
    First->data = A[0];
    First->next = NULL;
    last = First;
    for (i = 1; i < n; i++)
    {
        t = (struct Node*) malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}
```

```
void create 2 (int A[], int n)
```

```
{
    Second = (struct Node*) ...
    Second->data = A[0];
    Second->next = NULL;
    last = Second;
}
```


Page No. _____
Date: / /

```
void Concat (struct Node *P, struct Node *q)
```

```
{
    third = P;
    while (P->next != NULL)
        P = P->next;
    P->next = q;
}
```

```
void Merge (struct Node *P, struct Node *q)
```

```
{
    struct Node *last;

    IF (P->data < q->data)
    {
        third = last = P;
        P = P->next;
        third->next = NULL;
    }
```

```
    else
    {
        third = last = q;
        q = q->next;
        third->next = NULL;
    }
```

```
    while (P && q)
```

```
    {
        IF (P->data < q->data)
        {
            last->next = P;
```

```
            last = P;
            P = P->next;
            last->next = NULL;
```

```
        else
```

```
        {
            last->next = q;
```

```
            last = q;
            q = q->next;
```

```
            last->next = NULL;
```

```
        }
    }
    IF (P) last->next = P;
    IF (q) last->next = q;
```

int main()

{

int A[] = {10, 20, 30, 40, 50};

int B[] = {5, 15, 25, 35, 45};

create(A, 5);

create(B, 5);

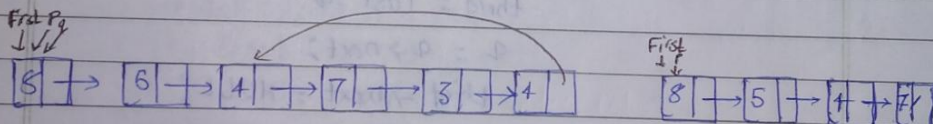
Concat(First, Second);

Display(Third) → 10, 20, 30, 40, 50, 5, 15, 25, 35, 45

Merge(First, Second);

Display(Third) → 5, 10, 15, 20, 25, 30, 35, 40, 45, 50

Check For Loop in LinkedList



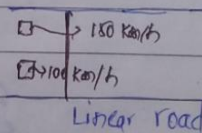
LOOP

Floyd's cycle
Finding

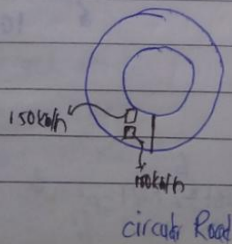
Algorithm → To check for loop logic

Linear

To check for linear
it's last pointer is
null.



These two car
never goint
to meet



These two car will
surely meet.

Make P move one step & q move 2 step

```

int isloop (Node *f)
{
    Node *P, *q;
    P = q = f;
    do
    {
        P = P->next;
        q = q->next->next;
        // First check
        // q reached NULL or not
        if (q == NULL) return false;
        while (P != q)
        {
            P = P->next;
            q = q->next->next;
        }
        return P == q ? true : false;
    } while (1);
}

```

IF (P == q) return true; else return false;

→ int main()

```

{
    struct Node *t1, *t2;
    int A[] = {10, 20, 30, 40, 50};
    create(A, 5);
}

```

t1 = First->next->next;

t2 = First->next->next->next->next;

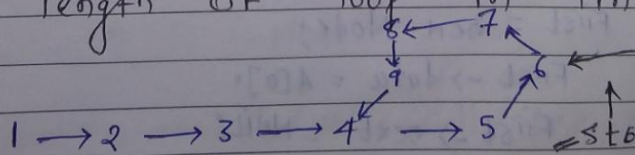
t2->next = t1;

Pointing on 30

Pointing on 50

Forming loop

Find length of loop in linkedlist



stop one pointer where both meet
move second pointer
& count the step

Let's code a c++ class For Linked List

```
Class Node
{
    Public:
        int data;
        Node *next;
};
```

```
Class Linked List
```

```
{
    Private:
        Node *first;
    Public:
        LinkedList() { first = NULL; }
        LinkedList (int A[], int n)
        ~ LinkedList()
        void Display()
        void Insert (int index, int x)
        int Delete (int index)
        int length();
};
```

```
LinkedList::LinkedList (int A[], int n)
```

```
{
    Node *last, *t;
    int i=0;
```

```
    first = new Node;
```

```
    first->data = A[0];
```

```
    first->next = NULL;
```

```
    last = first;
```


Page No. _____
Date: / /

```
For(i=1; i<n; i++)
```

```
{
```

```
    t = new Node;
```

```
    t->data = A[i];
```

```
    t->next = NULL;
```

```
    last->next = t;
```

```
    last = t;
```

```
}
```

```
LinkedList :: ~LinkedList()
```

```
{
```

```
    Node *P = First;
```

```
    While(First)
```

```
    {
```

```
        First = First->next;
```

```
        delete P;
```

```
        P = First;
```

```
    }
```

```
}
```

```
void LinkedList :: Display()
```

```
{
```

```
    Node *P = First;
```

```
    While(P)
```

```
    { cout << P->data << " ";
```

```
      P = P->next;
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
int LinkedList :: Length()
```

```
{
```

```
    Node *P = First;
```

```
    if (index < 0 || index > Length())
```

```
        return;
```

```

t = new Node;
t->data = x;
t->next = NULL;

```

```

IF (index == 0)
{
    First = t;
}
else
{
    For (int i = 0; i < index - 1; i++)
        P = P->next;
    t->next = P->next;
    P->next = t;
}

```

```

int LinkedList::Delete (int index)

```

```

{
    Node *P, *Q = NULL;
    int x = -1;
    IF (index < 1 || index > Length())
        return -1;
    IF (index == 1)
    {
        P = First;
        First = First->next;
        x = P->data;
        delete P;
    }

```

```

    else
    {
        P = First;
        For (int i = 0; i < index - 1; i++)
        {
            Q = P;
            P = P->next;
        }
    }
}

```



```

l->next = P->next;
x = P->data;
delete P;

```

```

{
return x;
}

```

```

{

```

```

int main()

```

```

{
int A[] = {1, 2, 3, 4, 5};

```

```

Linked List l(1, 5);

```

```

l.Display(); // 1 2 3 4 5

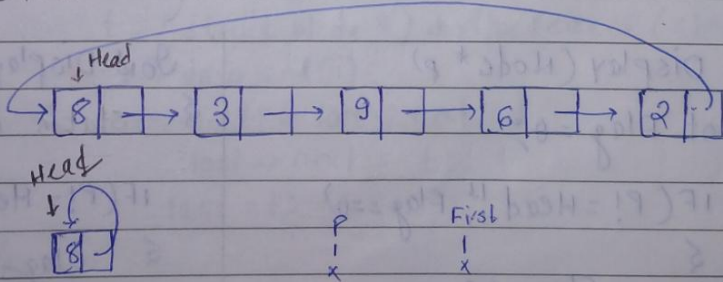
```

```

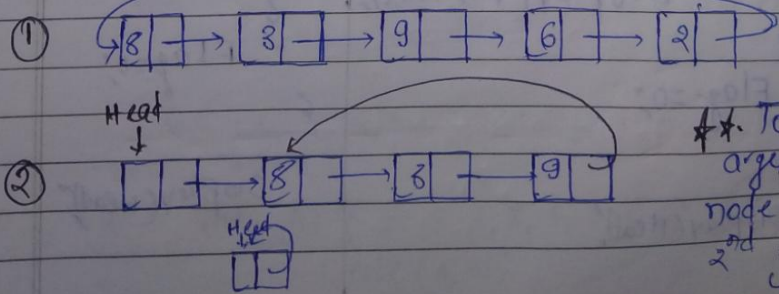
return 0;
}

```

Circular LinkedList

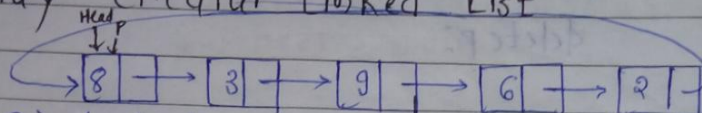


Two representation OF circular Linked List



** To support the argument that empty node should be circular 2nd representation is used

Display circular linked list



```

void Display(Node *P)
{
    while (P != Head)
    {
        printf("%d", P->data);
        P = P->next;
    }
    Display(Head);
}
  
```

it will not enter into loop

```

void Display(Node *P)
{
    do
    {
        printf("%d", P->data);
        P = P->next;
    } while (P != Head);
    Display(Head);
}
  
```

Using recursion

```

void Display(Node *P)
{
    static int Flag = 0;
    if (P != Head || Flag == 0)
    {
        Flag = 1;
        printf("%d", P->data);
        Display(P->next);
        Flag = 0;
    }
    Display(Head);
}
  
```

On each call, two variable will be created in stack

By using static, only one variable will be created each time in stack

```

void Display(Node *P)
{
    static int Flag = 0;
    if (P != Head || Flag == 0)
    {
        Flag = 1;
        printf("%d", P->data);
        Display(P->next);
        Flag = 0;
    }
    Display(Head);
}
  
```


→ #include <stdio.h>
#include <stdlib.h>

```
struct Node  
{  
    int data;  
    struct Node * next;  
}* Head;
```

void create (int A[], int n)

```
{  
    int i;  
    struct Node * t;  
    Head = (struct Node *) malloc (sizeof (struct Node));  
    Creating circular linked list  
    {  
        Head-> data = A[0];  
        Head-> next = Head;  
        last = Head;  
    }
```

For (i = 1; i < n; i++)

```
{  
    t = (struct Node *) malloc (sizeof (struct Node));  
    t-> data = A[i];  
    t-> next = last-> next;  
    last-> next = t;  
    last = t;  
}
```

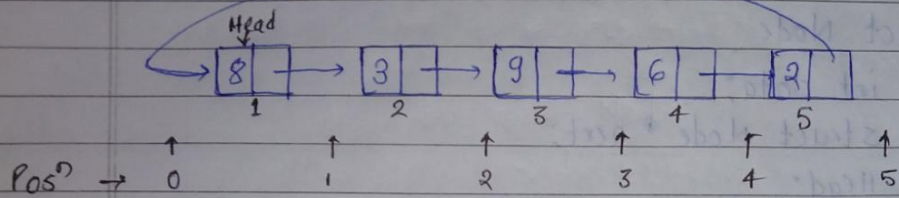
void Display (struct Node * h)

```
{  
    ,  
}
```

int main()

```
{  
    int A[] = { 2, 3, 4, 5, 6 };  
    create (A, 5);  
}
```

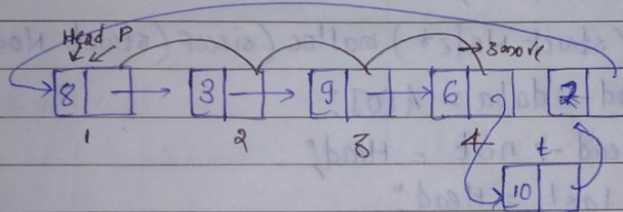
Inserting in a circular linked list



1. Insert before head

2. Inserting at any other position

Inserting at any other position



Min^o - $O(1)$

Max^o - $O(n)$

Node *t

Node *P = Head;

For (i=0; i < Pos-1; i++)

P = P->next;

t = new Node;

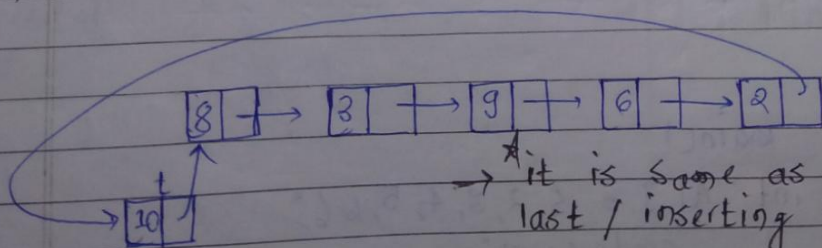
t->data = x;

t->next = P->next;

P->next = t;

same as linear
linked list

Insert before Head or Pos^o = 0



→ it is same as inserting at last / inserting before head

Node *P = Head;

Node *t = new Node;

t->data = x;

t->next = Head;

while (P->next != Head)

P = P->next;

P->next = t;

Head = t;

```
int length(struct Node*)  
{  
    int len = 0;  
    do  
        len++;  
    while (P != Head);  
    return len;  
}
```

Combining above two

void Insert (int pos, int x)

{

Node *t, *P;

if (index < 0 || index > length(P)) return;

if (pos == 0)

{

t = new Node;

t->data = x;

checking condⁿ IF Head is NULL

IF (Head == NULL)

{

Head = t;

Head->next = Head;

}

else

{

P = Head;

while (P->next != Head) P = P->next;

P->next = t; t->next = Head;

Head = t;

}

}

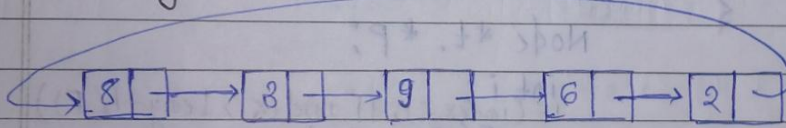
```

else
{
    P = Head;
    For(i = 0; i < Pos - 1; i++)
        P = P->next;

    t = new Node;
    t->data = x;
    t->next = P->next;
    P->next = t;
}

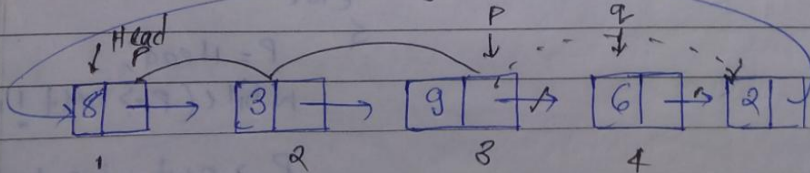
```

Deleting From circular Linked List



1. Deleting Head Node
2. Deleting a Node From given position

Delete Node at given position



$$Pos = 4$$

Method

without using
Tail pointer

P = Head;

For (i=0; i < Pos-2; i++)

{

P = P->next

* For deleting at + position

We have to move only

2 position

Min $\rightarrow O(1)$

Max $\rightarrow O(n)$

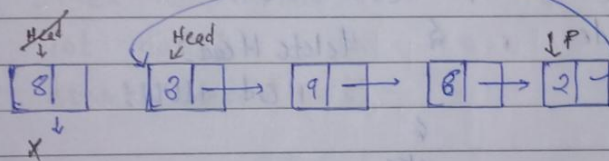
P->next = q->next;

x = q->data

delete q;

similar like
linear linked list

Deleting Head Node



P = Head;

While (P->next != Head)

P = P->next;

P->next = Head->next;

$O(n)$

x = Head->data;

delete Head

Head = P->next;

Combining above two

```
int Delete (int pos)
```

```
{
```

```
    int i
```

```
    Node *P, *q;
```

```
    IF (index < 0 || index > length (Head)) return -1
```

```
    IF (pos == 1)
```

```
{
```

```
        P = Head;
```

```
        while (P->next != Head)
```

```
            P = P->next;
```

```
        X = Head->data;
```

Handling case
of only 1 node
Present

```
    IF (P == Head)
```

```
{
```

```
        delete Head;
```

```
        Head = NULL;
```

```
    }
```

```
    else
```

```
{
```

```
        P->next = Head->next; delete Head; Head = P->next;
```

```
    }
```

```
    else
```

```
{
```

```
        P = Head;
```

```
        For (i = 0; i < pos - 2; i++)
```

```
            P = P->next;
```

```
            q = P->next;
```

```
            P->next = q->next;
```

```
            X = q->data;
```

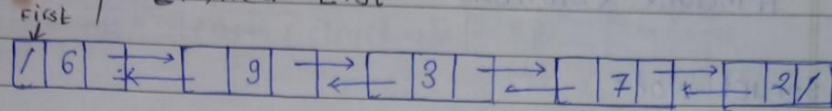
```
            delete q;
```

```
        }
```

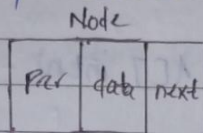
```
        return X;
```

```
    }
```


Doubly Linked List



IF we go on website & click link after link after that you move backward, any sitⁿ like this then there is use of doubly linked list



struct Node

```

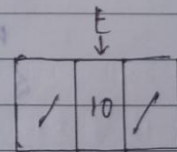
struct Node * prev;
int data;
struct Node * next;

```

```

→ struct Node * t;
t = new Node;
t->prev = NULL;
t->data = 10;
t->next = NULL;

```



★ Same operation that is performed on singly linked list can be performed on doubly linked list, only difference is that by using doubly linked list we can move in either direction.

Page No. _____
Date: / /

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
} * First = NULL;
```

Create Fⁿ → void create (int A[], int n)

```
{
    struct Node *t, *last;
    int i;
    First = (struct Node *) malloc (sizeof (struct Node));
    First->data = A[0];
    First->prev = First->next = NULL;
    last = First;
```

For (i=1; i<n; i++)

```
{
    t = (struct Node *) malloc (sizeof (struct Node));
    t->data = A[i];
    t->next = last->next;
    t->prev = last;
    last->next = t;
    last = t;
```

Display Fⁿ → void Display (struct Node *P)

```
{
    while (P)
    {
        printf ("%d", P->data);
        P = P->next;
    }
```


Page No. _____
Date: / /

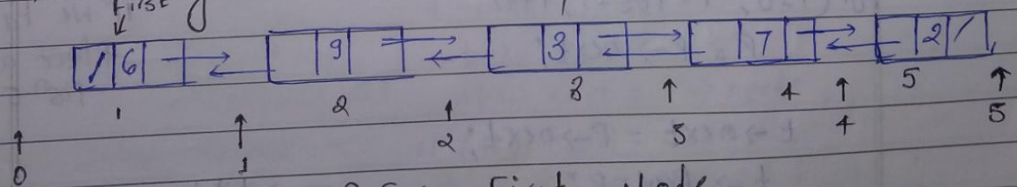
```

PrintF("\n"):
{
Find length → int Length(struct Node *p)
{
    int len = 0;
    While(p)
    {
        len++;
        p = p->next;
    }
    return len;
}

int main()
{
    int A[] = {10, 20, 30, 40, 50};
    Create(A, 5);
    PrintF("\n Length is: %d\n", Length(first)) → 5
    Display(first); → 10, 20, 30, 40, 50
    return 0;
}

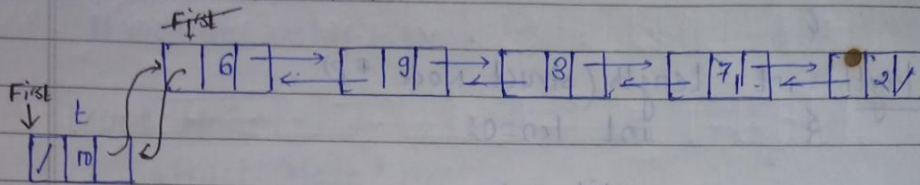
```

Inserting in a Doubly linked list



1. Before First Node
2. At any given index.

1. Before First node

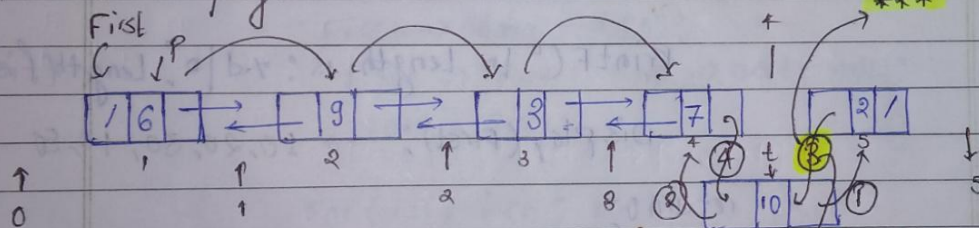


three links we have to modify
 Node *t = new Node;
 t->data = x;

* IF First = NULL, Only two links have to modify

t->Prev = NULL;
 t->next = First;
 First->Prev = t;
 First = t;

2. At any given index



Min - $O(1)$
 Max - $O(n)$ ← Node *t = new Node;
 t->data = x;

For (i=0; i<Pos-1; i++)
 P = P->next;

t->next = P->next;
 t->Prev = P;
 IF (P->next != NULL)
 P->next->Prev = t;

P->next = t;

I have to handle 3rd link very carefully, visualise IF we have to place at pos 5

*** For handling 3rd link IF we have to insert at index 5 then this operation will not be performed

Combining Above two
↓

Void insert (struct Node *P, int index, int x)

{

struct Node *t;

int i;

if (index < 0 || index > length(P))

return;

if (index == 0)

{

t = (struct Node *) malloc (sizeof(struct Node));

t->data = x;

t->prev = NULL;

t->next = First;

First->prev = t;

First = t;

}

else

{

for (i = 0; i < index - 1; i++)

P = P->next;

t = (struct Node *) malloc (sizeof(struct Node));

t->data = x;

t->prev = P;

t->next = P->next;

if (P->next)

P->next->prev = t;

P->next = t;

}

int main()

{

int

arr = {10, 20, 30, 40, 50};

create (arr);

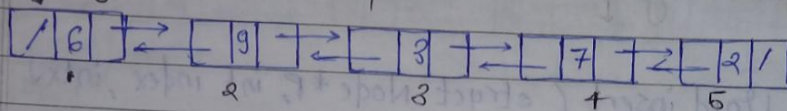
insert (First, 25);

Display (First);

return 0;

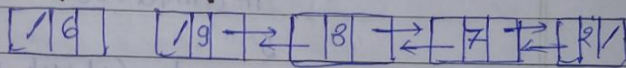
→ 10, 20, 25, 30, 40, 50

Deleting From doubly Linked List



1. Delete First Node
2. Delete From given index

1. Delete First Node



$P = \text{First};$

$\text{First} = \text{First} \rightarrow \text{next};$

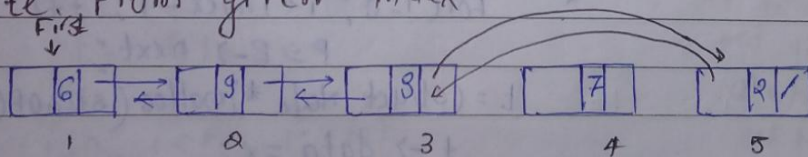
$x = P \rightarrow \text{data};$

delete $p;$

To check IF
First is not NULL

IF (First) $\text{First} \rightarrow \text{prev} = \text{NULL};$

2. Delete From given index



$\text{Pos} = 4$

$P = \text{First};$

For ($i = 0; i < \text{Pos} - 1; i++$)

$P = P \rightarrow \text{next};$

$P \rightarrow \text{prev} \rightarrow \text{next} = P \rightarrow \text{next};$

IF ($P \rightarrow \text{next}$)

$P \rightarrow \text{next} \rightarrow \text{prev} = P \rightarrow \text{prev};$

$x = P \rightarrow \text{data};$

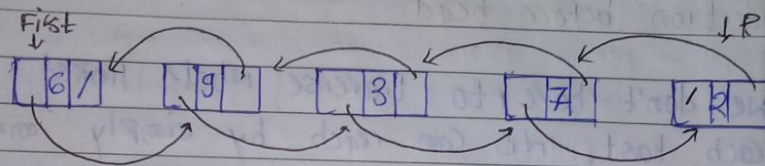
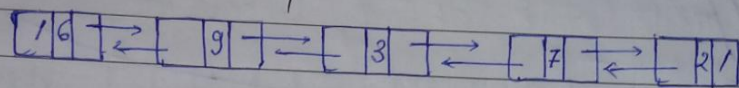
delete $P;$

Minⁿ $\rightarrow O(1)$

Maxⁿ $\rightarrow O(n)$

To check
IF $P \rightarrow \text{next}$
is present
or not

Reverse a Doubly linked list



see by
tracking

P = First; struct Node *temp;

while(P)

{

temp = P->next;

SNAPPING ← P->next = P->Prev;

P->Prev = temp;

P = P->Prev; ~~For moving~~

~~xx P! = NULL~~

to next node

during

reversing

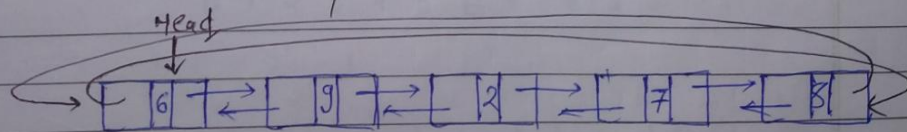
bringing
First to P

IF (P->next == NULL)

First = P;

}

Circular Doubly Linked List



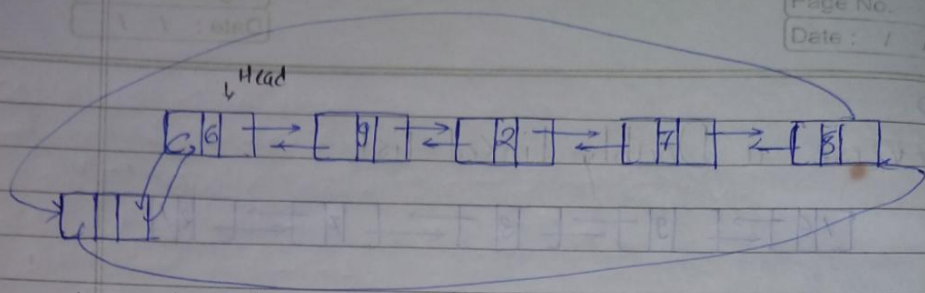
P = Head

do

\$ printf ("%d", P->data);

P = P->next;

while (P != Head)



Insertion before head

★ ★ We don't have to traverse whole linked list to reach last, we can reach by simply mentioning Head previous

O(1)

Complete code
↓

Insertion at any index

↓
same as previous

Delete in circular Linked List

↓
complete code

Compa

singly

circular singly

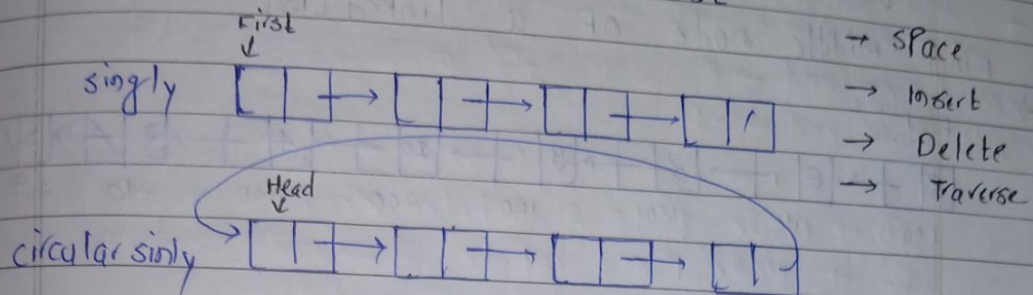
Doubly

circular Doubly

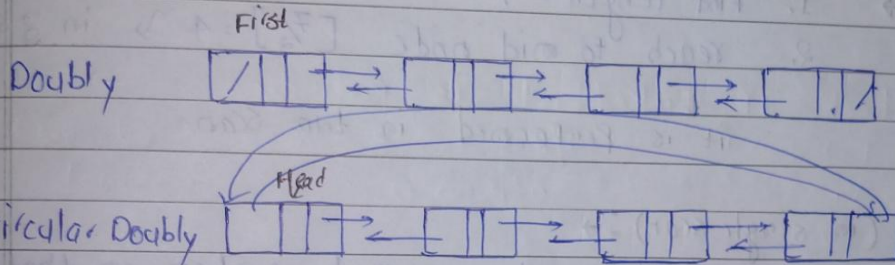
Compa

Area

Comparison OF Linked List

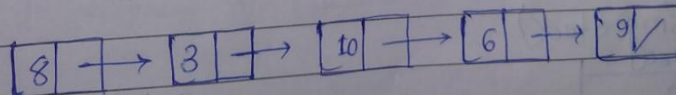
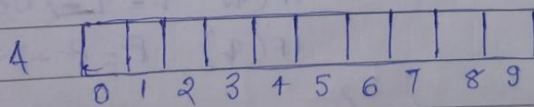


- Space
- Insert
- Delete
- Traverse



Comparison OF Array With Linked List

Array vs Linked List



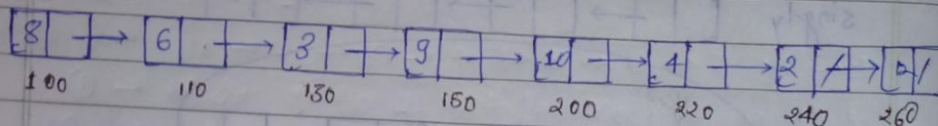
Sort
insertion sort
Merge sort

Student Challenge

Find Middle node of a Linked List

→

First $P \rightarrow Q$



M1 →

1. Find Length - 7

2. reach to mid node $[7/2] = 4 \rightarrow$ in 3 step

↓
it is performed in two scan

M2 (in single scan) →

* Move P by 1 step & Q by 2 step so that when Q reach at last P reach in middle

$P = Q = \text{First};$

While (Q)

{

$Q = Q \rightarrow \text{next};$

IF (Q) $Q = Q \rightarrow \text{next};$

IF (Q) $P = P \rightarrow \text{next};$

}

PrintF ("%d", P->data);

M3: → (Using stack)

260
240
220
200
150
130
110
100

$[8/2] = 4$

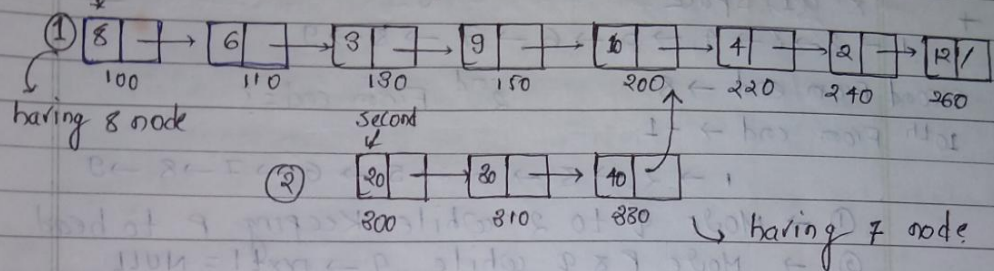
$[7/2] = 3$

$n + n/2 = 0(n)$

→ by counting the stack record

→ Stack of address

Find Intersection point of Two Linked List



→

	260		0
	240	260	+ 0
	220	240	+ 0
200	200	220	<u>300</u>
↓	150	200	0(0)
intersec ⁿ	130	330	
Point	110	310	
	100	300	

51

P = First;

while (P != NULL) Push(&stk1, P);

P = second;

while (P) Push(&stk2, P);

while (stackTop(stk1) == stackTop(stk2))

P = POP(&stk1); POP(&stk2);

6

1. N^{th} node from end of singly linked list in single pass
 → $\times O(1)$ space

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$

Second From end $\rightarrow 8$

2nd From end = ?

10th From end $\rightarrow -1$

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$

① $\rightarrow$ Move q to 2 while keeping p to head

② $\rightarrow$ Move $p \& q$ while $q \rightarrow \text{next} \neq \text{NULL}$

return $p \rightarrow \text{data}$

2. Check IF LinkedList is palindrome. $\rightarrow O(N)$

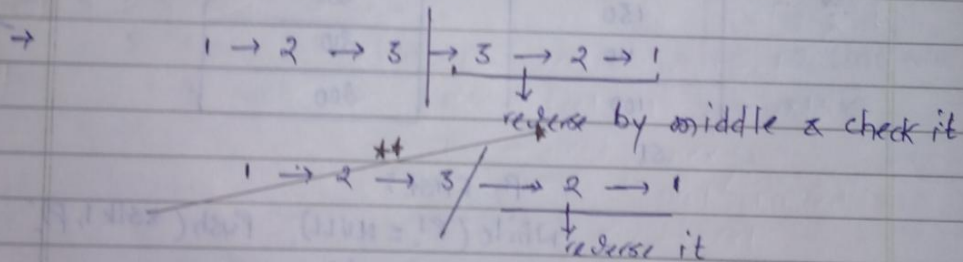
I/P $1 \rightarrow 2 \rightarrow 1$

I/P $\rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

$O(1)$

O/P $\rightarrow 1$

O/P $\rightarrow 0$



3. (i) Add 1 to a number represented as linked list

- (ii) Add two numbers represented by linked list

Hint: $\rightarrow$ Reverse linkedlist & do the operation

NOTE:- Always think of smaller test case For passing all test case (these forms corner case in pink)

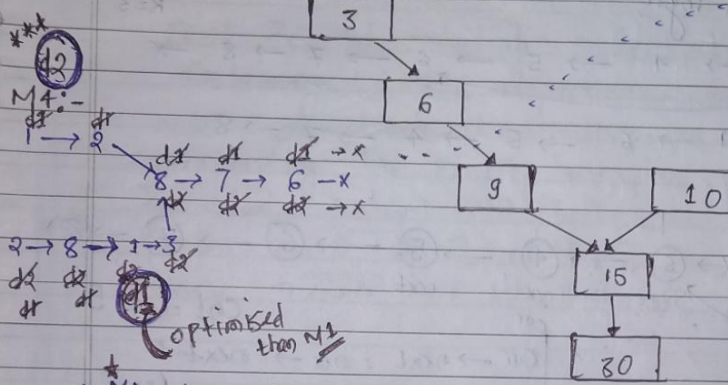
4. Intersection of Two linked lists

List 1 = 3 → 6 → 9 → common

List 2 = 10 → common

common = 15 → 30 → NULL

O/P → 15



d1 = head1, d2 = head2

while(d1 != d2)

if(d1 == NULL)

d1 = head1

else

d1 = d1 → next

if(d2 == NULL)

d2 = head2

else

d2 = d2 → next

return d1

O(N), O(1)

length1 → c1

length2 → c2

d = abs(c1 - c2)

now traverse bigger list till d step

traverse both list in parallel

M2 →

in one traverse multiply data by -1

second traverse return

linked = (Pointer → data)

O(N), O(1)

using stack

O(N), O(N)

at 15 → compare by address

Note:- In Q, like where result is very large use res % MOD

1000000007

ex:- Decimal equivalent of Binary linked list

1. Nth
→ x
1 → 2
Second From
10th From

Hard
5. Reverse linkedlist in groups of given size.
1 → 2 → 3 → 4 → 5 → 6 → 7 → 8
K = 4
O/P: → 4 2 2 1 8 7 6 5

①
②

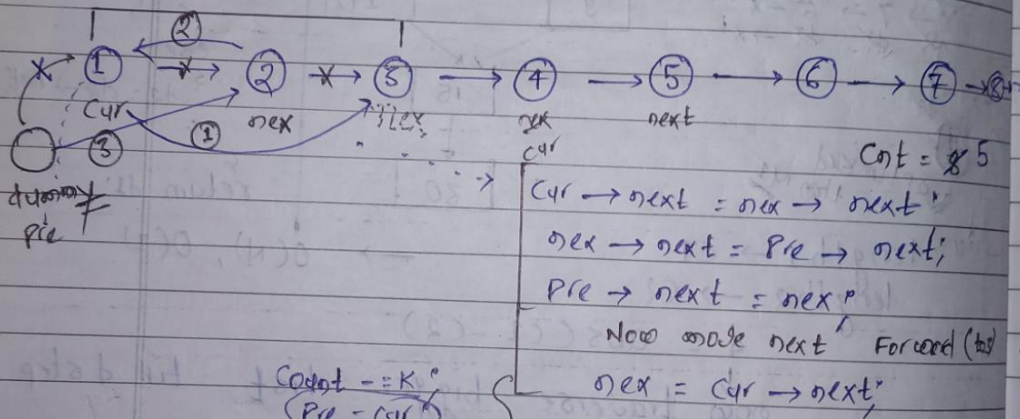
Form logic For revision

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → x
K=3

Resultant: 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8

2. Chec

I/P 1 -
O/P →



Count = K
(Pre = cur)

Again Perform above step to reverse next node

cur = Pre → nex;

nex = cur → nex

* cur is always standing at first of any group of size K, & nex is at second node

Hint

7. Remo

- * F

8. Find
scat

Hint:

26

OF given size.

5 → 4 → 8

K=3

7 → 8 →

7 → 8

⑥ → ⑦ → ⑧

cnt = 5

next → next;

pre → next;

next = next;

next Forward (to)

→ next;

node step to
node

at any given
node

6. Given a linked list of 0s, 1s, 2s, sort it

N=8

value[] = {5, 1, 2, 2, 1, 2, 0, 2, 2}

Output: 0 1 1 2 2 2 2 2

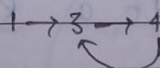
**

O(N)

* O(1)

→ Maintain 3 pointers named zero, one and two to point current ending nodes of linked list containing 0, 1, 2 respectively. For each traversed node, we attach it to the end of its corresponding list. Finally we link all three lists. To avoid any null checks, we use three dummy pointers zeroD, oneD, and twoD that work as dummy header of three list.

7. Remove loop in linked list



O/P → 1

→ Find start Node of the loop.

8. Find Middle element of linked list in just one scan

Hint: Make one pointer twice the speed of other pointer

28

Nth

X

1 → 2

and From

h From

①

②

Result

Check

1/P 1 -

0/P →

*

○

data

file

(i)

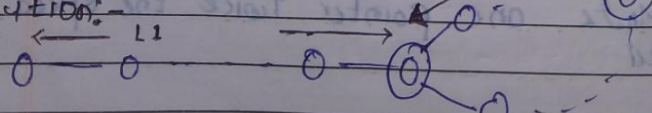
(ii)

H10

is loop exist

After meeting
slow & fast pointer
start moving head &
slow by one step
they will meet at starting of loop

in situation:-

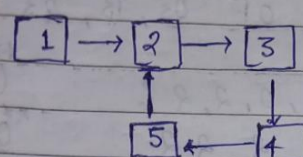


$$2(L1 + L2) = L1 + L2 + nC$$
$$L1 + L2 = nC$$
$$L1 = nC - L2$$

slow → $L1 + L2$
fast → $L1 + L2 + nC$

* total distance traveled by fast & slow

Make it every time during redirection.



To remove loop, all we need to do is to get pointer of last node (here it is 5)

we can use hashing
(check one by one) → we know that slow & fast ptr will meet at a common point at one of loop nodes (2 or 3 or 4 or 5 in a boxed diagram). Store the address of this in a pointer variable say ptr2. After that start from the head of the linked list and check for node one by one if they are reachable by ptr2.

Count the number of nodes in loop. Let the count be K. Fixed one pointer at head & another to a Kth node after head.

Made both pointers at same pace, they will meet at loop starting node. → Get last pointer with help of it.

during

Code of reverse in group of size k

```
ListNode* reverseKGroup(ListNode* head, int k) {
    if (head == NULL || k == 1) return head;
    ListNode* dummy = new ListNode(0);
    dummy->next = head;
    ListNode* curr = dummy, *nex = dummy, *pre = dummy;
    int count = 0;
```

```
while (curr->next != NULL) {
    curr = curr->next;
    count++;
```

```
while (count < k) {
```

```
    curr = pre->next;
```

```
    nex = curr->next;
```

```
    for (int i = 1; i < k; i++) {
```

```
        curr->next = nex->next;
```

```
        nex->next = pre->next;
```

```
        pre->next = nex;
```

```
        nex = curr->next;
```

```
    }
    pre = curr;
```

```
    count -= k;
```

```
    return dummy->next;
```

Input :-

1 → 2 → 3 → 4 → 5

Reversal steps:

2 → 1 → 3 → 4 → 5

3 → 2 → 1 → 4 → 5

4 → 3 → 2 → 1 → 5

5 → 4 → 3 → 2 → 1

is to get

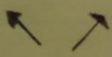
know that meet at
OF loop
in a body
his in a
tr2. After the
the linked list
one by one
by ptr2

OF nodes
nt be k
t head &
th node
after

t some
at loop
last point
it.

slow → 1 + 2
fast → 1 + 2
+ nC2

* total
distance by
fast is twice
of slow



Can also be solved using Merge two sorted list

Q. Flattening a Linked List

Given next pointer & bottom pointer

5 → 10 → 19 → 28

1

7

1

8

1

30

20

22

50

35

40

45

row and column are sorted, arrange them in a single linked list in sorted order

Result

O/P: 5 → 7 → 8 → 10 → 19 → 20 → 22 → 28 → 30 → 35 → 40 → 45 → 50

Brute Force → $O(N*M)$, $O(N*M)$

↓

optimized $O(N*M)$

```
struct Node* mergeTwoLists(struct Node* l1, struct Node* l2)
{
    struct Node* res = NULL;
```

```
    if (l1 == NULL) return l2;
```

```
    if (l2 == NULL) return l1;
```

```
    if (l1->data > l2->data)
```

```
    { std::swap(l1, l2); }
```

```
    res = l1;
```

```
    while (l1 != NULL && l2 != NULL)
```

```
    {
        Node* temp = NULL;
```

```
        while (l1 != NULL && l1->data <= l2->data)
```

```
        {
            temp = l1;
```

```
            l1 = l1->bottom;
```

```
        }
        temp->bottom = l2;
```

```
        std::swap(l1, l2);
```

```
    }
    return res;
```

Node* Flattening struct

Q1 Merge

[5] → [3]

ing Merge

on are
if then
single linked
sorted order

→ → →

(N*N)

struct Node*

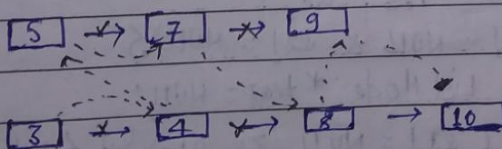
Q1

Merge two sorted linked list

Brute Force $\rightarrow O(n), O(n)$

↓

optimize it [inplace] $\rightarrow O(n), O(1)$



Q2

Q3

```

Node* Flatten (Node* root)
{
    struct Node * curr = root, * prev = root, * q = NULL, * P = NULL;
    while (curr != NULL)
    {
        curr = curr->next;
        prev = mergeTwoLists (prev, curr);
        P = prev;
    }
    while (P != NULL)
    {
        q = P;
        P = P->bottom;
        q->next = P;
    }
    return prev;
}
  
```

$l1, l2$

$[5] \rightarrow [7] \rightarrow [9] \rightarrow \text{NULL}$

res $\rightarrow [3] \rightarrow [4] \rightarrow [8] \rightarrow [10] \rightarrow \text{NULL}$

l1

make $l1$ whichever smaller

temp = NULL

$l2$
 $[5] \rightarrow [7] \rightarrow [9]$

res $\rightarrow [3] \rightarrow [4] \rightarrow [8] \rightarrow [10]$

temp = NULL

temp = $l1$, $l1 = l \rightarrow \text{next}$ (till when $l1$ is smaller)

temp $\rightarrow \text{next} = l2$

(Now swap ($l1, l2$) whichever is smaller)

after swapping

$l1$
 $l2$
 $[5] \rightarrow [7] \rightarrow [9] \rightarrow \text{NULL}$

res $\rightarrow [3] \rightarrow [4] \rightarrow [8] \rightarrow [10] \rightarrow \text{NULL}$

Now repeat above thing

IF ($l1 == \text{NULL}$) return $l2$

IF ($l2 == \text{NULL}$) return $l1$

IF ($l1 \rightarrow \text{val} > l2 \rightarrow \text{val}$) std :: swap($l1, l2$)

ListNode* res = $l1$

while ($l1 != \text{NULL} \& \& l2 != \text{NULL}$)

ListNode* temp = NULL

while ($l1 != \text{NULL} \& \& l1 \rightarrow \text{val} < l2 \rightarrow \text{val}$)

temp = $l1$

$l1 = l1 \rightarrow \text{next}$

temp $\rightarrow \text{next} = l2$

std :: swap($l1, l2$)

return res

6

Q. Add

Q. Co

$O(N), O(2)$

Q. Add Two No.s represented as linked list

ListNode *head1 = NULL, *Pre1 = NULL, *t = NULL;

while (d1 != NULL || d2 != NULL || sum)

sum = (d1 != NULL ? d1->val : 0) + (d2 != NULL ? d2->val : 0)

t = (ListNode*) malloc (sizeof (ListNode));

t->val = sum % 10;

t->next = NULL;

sum /= 10;

IF (head1 == NULL) { head1 = t, Pre1 = t;

else { Pre1->next = t, Pre1 = t;

}

IF (d1) { d1 = d1->next;

IF (d2) { d2 = d2->next;

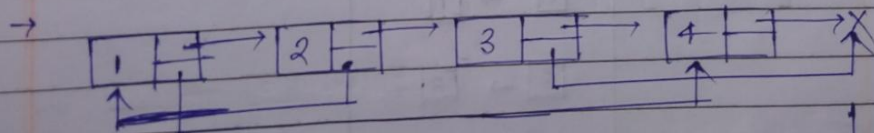
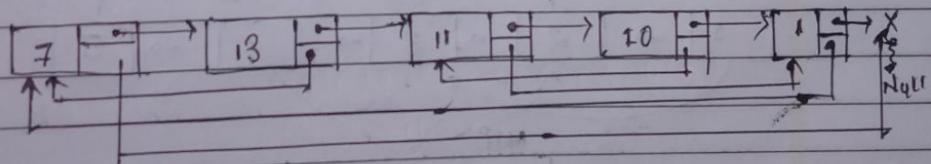
return head1;

(E1) when
d1 is
smaller)

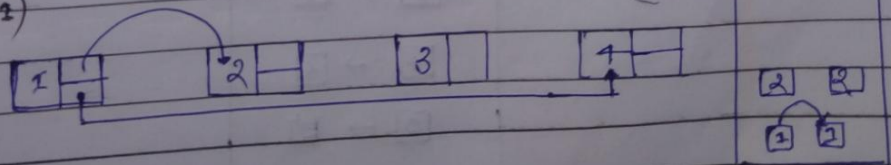
whichever is

next

Q. Construct a Deep Copy



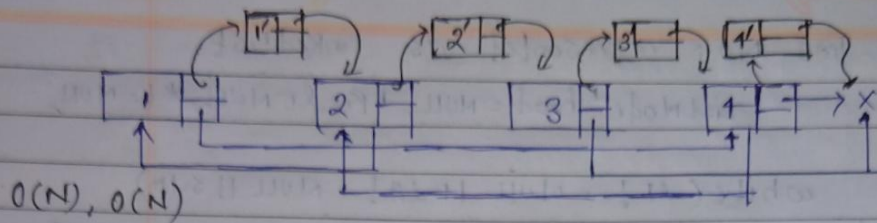
$O(N)$, $O(1)$



P->next

1.
→

Second
10th



Optim

Use map $\Rightarrow$
 $O(n), O(n)$

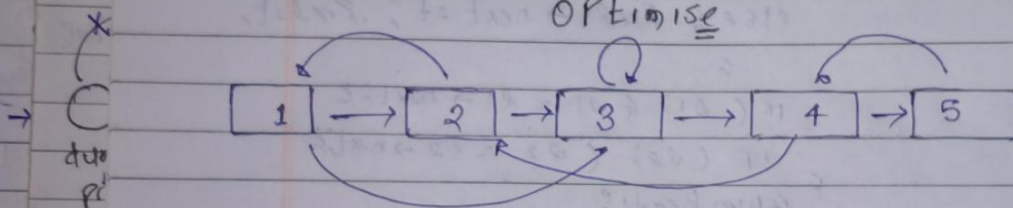
created
map
using
link

2.

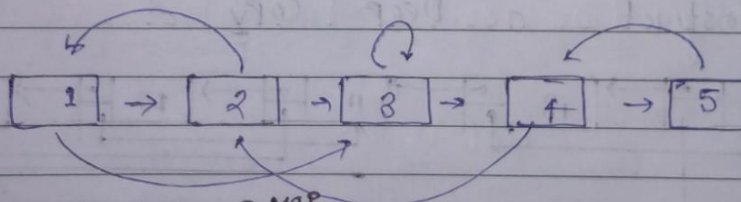
Result

⇓

Optimise



Copy



- map

1	→	1
2	→	2
3	→	3
4	→	4
5	→	5

3.

Optimise

Random Pointer

$O(N)$

created
map
using
link

Now solve above

next

++
→ cor)
"y d" P→
next
"0" /

Resu

द्वि

0	
1	
2	
3	
4	

Sparse Matrix using LinkedList

Array of LinkedList

	0	1	2	3	4	5
0	→ [1 8 1]	→ [0 0 0]	→ [0 8 0]	→ [0 8 0]	→ [0 8 0]	→ [0 8 0]
1	→ [3 7 1]	→ [1 0 0]	→ [0 7 0]	→ [0 0 0]	→ [0 0 0]	→ [0 0 0]
2	→ [0 5 1]	→ [2 5 0]	→ [0 0 9]	→ [0 0 9]	→ [0 0 9]	→ [0 0 9]
3	→ [5 3 1]	→ [3 0 0]	→ [0 0 0]	→ [0 0 3]	→ [0 0 3]	→ [0 0 3]
4	→ [0 4 1]	→ [4 6 0]	→ [0 4 0]	→ [0 4 0]	→ [0 4 0]	→ [0 4 0]

Node *A[∞];

A[0] = new Node;

Node	col	val	next

struct Node

int col;
int val;

struct Node *next;

student Exercise

1. Create sparse Matrix
2. Display sparse Matrix
3. Add sparse Matrix

2. Display sparse Matrix :- HINT

For (i=0; i<∞; i++)

if (i==0; i<h; i++)

if (j==0; j<w; j++)

if (j==P->col)

PrintF("%d", P->val);

P = P->next;

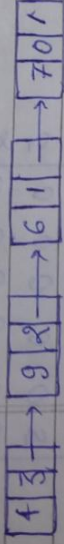
else PrintF("0");

PrintF("\n");

	0	1	2	3	4
0	→ [1 8 1]	→ [3 7 1]	→ [0 5 1]	→ [5 3 1]	→ [0 4 1]
1	→ [3 7 1]	→ [1 0 0]	→ [0 7 0]	→ [0 0 0]	→ [0 0 0]
2	→ [0 5 1]	→ [2 5 0]	→ [0 0 9]	→ [0 0 9]	→ [0 0 9]
3	→ [5 3 1]	→ [3 0 0]	→ [0 0 0]	→ [0 0 3]	→ [0 0 3]
4	→ [0 4 1]	→ [4 6 0]	→ [0 4 0]	→ [0 4 0]	→ [0 4 0]

Polynomial representation using Linked list

$$P(x) = 4x^3 + 9x^2 + 6x + 7$$



Node		
Coeff	Exp	next

Evaluation

double Eval(int x)

double sum = 0.0;

Node * q = P;

while (q != NULL)

{

sum += q->coeff * pow(x, q->exp);

q = q->next;

}

return sum;

}

student exercise

① Adding two polynomials


```
→ #include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
struct Node
```

```
{
    int coeff;
```

```
    int exp;
```

```
    struct Node *next;
```

```
}; *poly = NULL;
```

```
void create()
```

```
{
    struct Node *t, *last = NULL;
```

```
    int num, i;
```

```
    printf("Enter number of terms");
```

```
    scanf("%d", &nym);
```

```
    printf("Enter each term with coeff and exp");
```

```
    for(i=0; i<nym; i++)
```

```
    {
        t = (struct Node *) malloc(sizeof(struct Node));
```

```
        scanf("%d %d", &t->coeff, &t->exp);
```

```
        t->next = NULL;
```

```
        if(poly == NULL)
```

```
        {
            poly = t; last = t;
```

```
        }
        else
```

```
        {
            last->next = t;
```

```
            last = t;
```

```
        }
```

```
    }
```

```
void Display (struct Node *P) {
```

```
    while(P)
```

```
    { printf("%d\t%d\t", P->coeff, P->exp);
      P = P->next;
    }
```

```
    printf("\n");
```

```
long Eval (struct Node *P, int x)
```

```
{ long val = 0;
```

```
    while(P)
```

```
    { val = P->coeff * Pow(x, P->exp);
      P = P->next;
    }
```

```
    return val;
```

```
int main()
```

```
{
```

```
    create();
```

```
    Display(Poly);
```

```
    printf("%d\t%d\t", Eval(Poly, 1));
```

```
    return 0;
```

```
}
```


Page No.
 Date: / /

Date: / /
 Page No.